CPD first project

Deadline 23 March – 23:59
The students can organize into groups of up to 3 people.

1 Environment

The goal of this project is to understand how services available in cloud computing platforms can be used for creating applications that are scalable, fast, and highly available.

The project will consist in the design and implementation of the backend for a hospital system like hospital electronic health record databases as in the MIMIC-III database, and companion testing scripts.

The system will manage the hospital data and allow new patients. Users can also pose a question to the doctor. A question can only be answered by the doctor that treated the patients, and there can be only one answer for each question.

For implementing its features, the system must maintain, at least, the following information:

- PATIENTS: Stores patient demographic details like patient ID, date of birth, and gender.
- Media: manages images and videos used in the system, for example patient ID images;
- ADMISSIONS: in MIMIC-III, an admission (hospital stay) represents a medical event. It has an owner (patient), start/end times, and status (e.g., active/discharged)
- Progress: Medical intervention or test result. Medications and other input events (INPUTEVENTS_MV) and lab tests (LABEVENTS) track changes in a patient's status.
- Questions: questions and replies. Each question must include the Patient it refers to, the user that posed the question and the text of the message.

The system must support, at least, the following basic operations using standard REST rules for defining endpoints:

- USERS (/rest/user): create patient, delete patient, update patient. After a patient is deleted, ADMISSIONS and Progress from the user appear as been performed by a default "Deleted User" user.
- MEDIA: upload media, download media. (you can choose any type of data)
- ADMISSIONS: create, update a medical event.
- PROGRESS: Medical intervention or test result.
- Questions (/rest/patients/id/question): create question or reply, list questions. ID of the patients.

Operations that create an entity with an associated media object (e.g., users and Progress) can be implemented either by having clients executing two consecutive calls, the first for uploading the image and the second for the corresponding operation; or by executing a single operation that includes the media and the remaining data.

2 Environment

The project will be implemented in GCP, using the resources available to students under the GCP student program. Check the documentation in lab 1 for more information on how to claim the credits for you GCP account. NOTE: GCP services have associated costs. Be careful with the configuration of your services and the cost when you leave services/resources running, specially, those that are more expensive.

3 Base version

The base version consists in the basic version of the backend service, that includes the base REST endpoints associated with PATIENTS, Media, ADMISSIONS, Progress, Questions.

You must decide how the service information is stored in the underlying storage services / database services, creating the necessary objects/documents.

Besides the base endpoints associated with the previous resources, the service should provide the following endpoint:

- List of patients with longer waiting times;
- List of a patient Progress

3.1 Services to use

The system should use the following GCP services:

- AppEngine: for creating the REST servers (see lab 1 for more info).
- Blob storage: for storing BLOB (see lab 2 for more info).
- DB: for storing stucture data (lab 1)

3.2 FaaS

Use Google Cloud functions to implement some functionality in your application. Suggestion: Use Google Cloud functions to perform some periodic computation (e.g., update longer waiting times, garbage-collection). Note: Using Google Functions to solve the next feature does not count.

3.3 Testing

You should develop the necessary scripts for testing your backend service. Please deploy your system in multiple location to test the different performance metrics. You can implement using your own system or https://github.com/artilleryio/artillery

4 Report and Evaluation

The project report should present the work developed in a clear, concise and complete way (suggested maximum dimension: 6 pages in a 11pt font, A4 with decent margins). The report should not include code, but can include pseudo-code to help presenting some design features. The following structure is suggested:

- Introduction: briefly explain the context and goal of the system being developed;
- Design: introduce the design of the system, including the architecture with its main components and how these components interact for implementing the different features of the system;
- Implementation: briefly introduce any implementation detail that you think it is worth being highlighted;
- Evaluation: evaluation results (and discussion) if needed using artillery.
- Conclusions The report can be written in Portuguese or English.